AI-Assisted Multi-Stage Framework for Source Code Vulnerability Detection

Project Report submitted in the partial fulfilment

of

Bachelor of Technology Integrated

In

Computer Engineering

by

Manav Gangar (C109)
Krisha Chawan (C082)
Jude Dsouza(C116)
Neel Vora (C109)

Under the supervision of

Prof Sofia Francis

(Assistant Professor, Computer Engineering, MPSTME)

SVKM's NMIMS University

(Deemed-to-be University)

MUKESH PATEL SCHOOL OF TECHNOLOGY MANAGEMENT & ENGINEERING (MPSTME) Vile Parle (W), Mumbai-56

(2026-2027)

CERTIFICATE

This is to certify that the project entitled (AI-Assisted Multi-Stage Framework for Source Code Vulnerability Detection), has been done by Mr. Manav Gangar, Ms. Krisha Chawan, Mr. Jude Dsouza and Mr. Neel Vora under my guidance and supervision & has been submitted in partial fulfilment of the degree of (name of the program) in (name of the stream) of MPSTME, SVKM's NMIMS (Deemed-to-be University), Mumbai, India.

Sofia Francis (PS). SOFIA

Project mentor (name and Signature)

Examiner (name and Signature) (Intenal Guide)

Date 24.10.26.

Place: Mumbai

(HoD) (name and Signature)

ACKNOWLEDGEMENT

Student Names

NAME	ROLL NO.	SAP ID
Manav Gangar	C109	70322100089
Krisha Chawan	C082	70322100171
Jude Dsouza	C116	70322100197
Neel Vora	C108	70322100049

ABSTRACT

Software vulnerabilities pose significant security risks, potentially resulting in data breaches, unauthorized access, and system compromise. Traditional Static Application Security Testing (SAST) tools rely on predefined rules to detect insecure coding practices but may generate false positives and provide limited remediation guidance. This project, AI-Assisted Software Vulnerability Detection, proposes an integrated approach combining static analysis, machine learning, and generative artificial intelligence to improve vulnerability detection and explainability.

The system utilizes Semgrep and Bandit to identify potentially insecure code patterns and a fine-tuned CodeBERT model to classify code as vulnerable or non-vulnerable. Retrieval-Augmented Generation (RAG) retrieves relevant security knowledge from resources such as the CWE Top 25 and OWASP Top 10, providing contextual information to a Large Language Model (LLM). The LLM generates understandable vulnerability explanations and suggests potential remediation strategies through a web-based interface.

By integrating multiple detection techniques with contextual security knowledge and AI-generated guidance, the proposed system aims to help developers identify, understand, and address potential software vulnerabilities more effectively. Its performance can be assessed using metrics such as accuracy, precision, recall, and F1-score. The system is intended to support secure software development, while its findings and suggested fixes require appropriate validation.

Table of Contents

Topics	Page
List of Figures	i
List of Tables	ii
Abbreviations	iii
Chapter 1: Introduction	
1.1 Background of the project topic	11
1.2 Motivation and scope of the report	12
1.3 Problem Statement	13
1.4 Salient Contribution	14
1.5 Organization of Report	15
Chapter 2: Literature survey	
2.1 Introduction to the Overall Topic	17
2.2 Exhaustive Literature Survey	18
2.3 Comparative Analysis of Existing Research	23
2.4 Research Gap Identification	24
2.5 Chapter Summary	26
Chapter 3 Methodology and Implementation	
3.1 Block diagram	27
3.2 Hardware description	27
3.3 Software description, flowchart / algorithm	28
3.4 Chapter Summary	34
Chapter 4 Result and Analysis	
4.1 Introduction	35
4.2 Dataset and Evaluation Setup	35
4.3 Static Application Security Testing (SAST)	36

4.4 CodeBERT Classifier and Metrics 37

4.5 Confusion Matrix and Error Analysis 38

4.6 Retrieval-Augmented Generation (RAG) Retrieval Evaluation 39

4.7 Combined Interpretation of the Framework 41

4.8 Validity Considerations 41

4.9 Chapter Summary 42

Chapter 5 Advantages, Limitations and Applications

5.1 Introduction 43

5.2 Advantages 43

5.3 Limitations 44

5.4 Applications 45

5.5 Chapter Summary 47

Chapter 6 Conclusion and Future Scope

6.1 Introduction 48

6.2 Conclusion 48

6.3 Future Scope 49

6.4 Chapter Summary 51

References 53

LIST OF FIGURES

Figure No.	Title	Page
Figure 1	Source Code Submission for Developers	26
Figure 2	Security Analysis Results Dashboard	27
Figure 3	Static Application Security Testing (SAST) Findings	27
Figure 4	AI-Generated Security Recommendations and Remediation Guidance	28
Figure 5	Retrieved Cybersecurity Knowledge Using Retrieval-Augmented Generation (RAG)	29
Figure 6	System Block Diagram	22
Figure 7	System Flowchart: AI-Assisted Software Vulnerability Detection	25

LIST OF TABLES

Table No.	Title	Page
Table 1	Student Names and Identification Details	3
Table 2	Hardware Configuration	23
Table 3	Software Requirements and Technologies	24
Table 4	Dataset Split and Class Distribution	31
Table 5	Framework Components and Their Purpose	31
Table 6	Classifier Evaluation Metrics	32
Table 7	Confusion Matrix	32
Table 8	RAG Retrieval Evaluation Metrics	33

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BGE	BAAI General Embedding
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
FAISS	Facebook AI Similarity Search
F1-score	Harmonic mean of precision and recall
LLM	Large Language Model
ML	Machine Learning
OWASP	Open Worldwide Application Security Project
RAG	Retrieval-Augmented Generation
ROC-AUC	Area Under the Receiver Operating Characteristic Curve
SAST	Static Application Security Testing
CI/CD	Continuous Integration / Continuous Deployment
RAM	Random Access Memory
UI	User Interface

Chapter 1
Introduction

Software security is an important aspect of modern software development because vulnerabilities in source code can lead to data breaches, unauthorized access, and system compromise. As software applications become more complex, identifying and correcting security weaknesses at an early stage becomes increasingly important. Traditional security techniques, such as manual code reviews and Static Application Security Testing (SAST), help identify insecure coding patterns, but their findings may require considerable human effort to understand and resolve.

The proposed project, AI Security Copilot for Software Vulnerability Detection, aims to develop an AI-assisted framework that combines static code analysis, machine learning, Retrieval-Augmented Generation (RAG), and Large Language Models (LLMs). The system uses Semgrep and Bandit to identify potentially insecure code patterns and a fine-tuned CodeBERT model to classify source-code functions as vulnerable or non-vulnerable. It also retrieves relevant cybersecurity information from the Common Weakness Enumeration (CWE) knowledge base to support understandable vulnerability explanations and remediation recommendations.

An important feature of the proposed framework is its patch verification process. After a potential fix is generated, the modified code can be scanned again to check whether the original security issue persists. The overall objective is to help developers identify, understand, and address software vulnerabilities through a unified workflow rather than relying only on conventional detection tools.

The project integrates these techniques into a multi-stage security analysis pipeline. Its intended outcome is to improve the efficiency of vulnerability analysis and provide developers with more useful, context-aware security guidance. However, generated fixes must be independently validated before being accepted or deployed.

1.1 Background of the project topic

Software vulnerabilities are weaknesses in software code that attackers may exploit to compromise the confidentiality, integrity, or availability of a system. Common examples include improper input validation, insecure handling of user data, and injection vulnerabilities. Identifying such weaknesses during software development helps reduce security risks and the cost of correcting defects.

Traditional vulnerability detection methods include manual code review and Static Application Security Testing (SAST). SAST tools examine source code without executing the application and use predefined rules to identify potentially insecure coding practices. Tools such as Semgrep and Bandit are useful for identifying known vulnerability patterns. However, their findings may include false positives, and developers may need to spend additional time understanding the reported issues and deciding how to fix them.

Recent developments in artificial intelligence have introduced machine learning models that can analyze source code and identify patterns associated with vulnerabilities. CodeBERT is a pretrained model designed to understand programming-language and natural-language representations. Research has explored its application to vulnerability classification, demonstrating the potential of transformer-based models for source-code security analysis [1].

In addition, Retrieval-Augmented Generation (RAG) enables an AI system to retrieve relevant information from a cybersecurity knowledge base before generating an explanation. Resources such as MITRE's Common Weakness Enumeration (CWE) provide information about software weaknesses, their causes, and possible mitigation strategies. Combining this information with a Large Language Model can help generate explanations and remediation recommendations that are more relevant to the detected issue [2].

Based on these developments, the proposed AI Security Copilot combines static analysis, CodeBERT-based vulnerability classification, cybersecurity knowledge retrieval, AI-assisted remediation, and patch verification. The framework is designed to support developers throughout the vulnerability management process, from identifying a potential weakness to checking whether a proposed correction resolves it.

1.2 Motivation and scope of the report

The primary motivation behind this project is to address the limitations of existing software vulnerability detection approaches. Traditional tools can identify suspicious code patterns, but developers still need to investigate the findings, understand their security implications, and implement suitable corrections. This process can be time-consuming, particularly when a project contains a large amount of source code.

Furthermore, machine learning models can provide additional vulnerability classification capabilities, while generative AI can help explain security issues in simpler language. However, AI-generated explanations and code fixes cannot automatically be assumed to be correct. Therefore, combining multiple detection techniques with security-specific knowledge and a verification stage is an important motivation for developing the proposed framework.

The main motivations are:

1. To reduce the manual effort required to analyze software vulnerabilities.
2. To combine rule-based static analysis with machine learning-based vulnerability classification.
3. To provide understandable explanations of detected security weaknesses.
4. To use cybersecurity references to generate context-aware remediation recommendations.
5. To verify proposed code fixes by performing security scans again.
6. To support developers in following a more systematic and efficient secure software development process.

The scope of this project covers the design, implementation, and evaluation of an AI-assisted framework for source-code vulnerability detection and remediation support.

The project includes the following components:

1. Static Code Analysis: Semgrep and Bandit are used to identify potentially insecure coding patterns.
2. Vulnerability Classification: A fine-tuned CodeBERT model classifies Python source-code functions as vulnerable or non-vulnerable.

3. Dataset Preprocessing: The PyCode-Vul dataset is cleaned to remove missing and duplicate samples and to reduce data leakage between dataset splits.
4. Security Knowledge Retrieval: A RAG module retrieves relevant information from MITRE CWE version 4.20 using BGE-small-en-v1.5 embeddings and FAISS similarity search.
5. AI-Assisted Explanation and Remediation: A Large Language Model uses detected findings and retrieved security knowledge to explain vulnerabilities and suggest potential fixes.
6. Patch Verification: The modified source code is scanned again to check whether the original vulnerability remains.
7. Result Presentation: A React-based dashboard is planned to present findings, explanations, suggested patches, and verification status.

The current research and evaluation primarily focus on Python source code and binary vulnerability classification. The framework is intended to assist developers; it does not eliminate the need for human review or independent validation of generated fixes.

1.3 Problem statement

Software vulnerability detection remains challenging because traditional security analysis tools may generate false positives, require manual investigation, and provide limited guidance for correcting identified weaknesses. Although machine learning and Large Language Models offer additional capabilities for analyzing code and generating remediation suggestions, their outputs may not always be accurate or secure.

Furthermore, vulnerability detection, explanation, remediation, and verification are often handled as separate activities. A proposed code fix may appear correct without actually eliminating the original vulnerability, making independent verification necessary.

Therefore, the problem addressed by this project is to design and develop an AI-assisted framework that integrates static application security testing, machine learning-based vulnerability classification, retrieval-augmented cybersecurity knowledge, AI-generated remediation recommendations, and automated patch verification into a unified workflow for source-code vulnerability detection.

The framework aims to help developers identify potential security weaknesses, understand their causes and impacts, obtain context-aware remediation suggestions, and verify whether the identified issues persist after modification. The project also seeks to evaluate the vulnerability classification component using performance measures such as accuracy, precision, recall, F1-score, and ROC-AUC.

1.4 Salient contribution

The salient contributions of the proposed project are as follows:

1. Integration of Multiple Vulnerability Detection Techniques: The framework combines Semgrep and Bandit with a fine-tuned CodeBERT classifier. This enables rule-based analysis and machine learning-based classification to contribute to the overall security analysis workflow.

2. Machine Learning-Based Vulnerability Classification: The project fine-tunes CodeBERT using the PyCode-Vul dataset to classify source-code functions as vulnerable or non-vulnerable. The dataset preprocessing process includes removing missing and duplicate samples and checking for overlapping samples across dataset splits.

3. Retrieval-Augmented Cybersecurity Knowledge: The framework incorporates a RAG module that retrieves relevant information from MITRE CWE version 4.20. It uses BGE-small-en-v1.5 embeddings and a FAISS vector index to find relevant security information that can support contextual explanations and remediation recommendations.

4. AI-Assisted Vulnerability Explanation and Remediation: The proposed system uses a Large Language Model to explain detected vulnerabilities, describe their possible impact, and suggest secure code alternatives. This is intended to make security findings easier for developers to understand and act upon.

5. Patch Verification Through Rescanning: Instead of treating an AI-generated fix as automatically correct, the framework includes a verification stage that reruns relevant security scans on modified code. This helps determine whether the original finding persists and provides evidence for the reported verification status.

6. Quantitative Evaluation of the Classifier: The research paper reports that the fine-tuned CodeBERT classifier achieved the following results on its evaluated test set:

    - Accuracy: 96.88%
    - Precision: 95.76%
    - Recall: 97.38%
    - F1-score: 96.56%
    - ROC-AUC: 99.33%

These results provide quantitative evidence of the classifier's performance on the evaluated dataset. The integration of detection, knowledge retrieval, remediation support, and verification represents the broader contribution of the proposed framework. These figures represent the classifier's reported evaluation, not the verified performance of the entire integrated framework.

1.5 Organization of report

The report is organized into six chapters, followed by references and appendices.

Chapter 1: Introduction

This chapter introduces the project, explains the background and motivation, defines the problem statement, describes the scope, and highlights the major contributions of the proposed framework.

Chapter 2: Literature Survey

This chapter reviews existing research on software vulnerability detection, static analysis, machine learning-based code classification, retrieval-augmented generation, and AI-assisted vulnerability remediation. It compares existing approaches and identifies the research gap addressed by the project.

Chapter 3: Methodology and Implementation

This chapter describes the proposed system architecture, dataset preprocessing, CodeBERT model training, static analysis tools, RAG implementation, AI-assisted remediation workflow, and patch verification process.

Chapter 4: Results and Analysis

This chapter presents and discusses the experimental results of the vulnerability classification model. It includes performance metrics, comparisons with baseline models, confusion matrices, and other relevant evaluation findings.

Chapter 5: Advantages, Limitations and Applications

This chapter discusses the benefits of the proposed framework, its limitations, and its potential applications in secure software development, code review, and vulnerability assessment.

Chapter 6: Conclusion and Future Scope

This chapter summarizes the work carried out, presents the main conclusions, and discusses possible future improvements, including support for additional programming languages, expansion of the cybersecurity knowledge base, and integration with continuous integration and continuous deployment pipelines.

The report concludes with the references cited throughout the document and appendices containing relevant supplementary material.

Chapter 2
Literature survey

2.1 Introduction to the Overall Topic

Software vulnerabilities, that is, weaknesses in source code that an attacker can exploit to compromise the confidentiality, integrity, or availability of a system, remain a persistent source of security and reliability risk. The techniques used to find them have developed through several overlapping stages: manual code review, rule-based Static Application Security Testing (SAST), machine-learning and deep-learning classifiers trained on la-beled code, pre-trained transformer models, and, most recently, Large Language Model (LLM) systems that also explain and repair what they find. Each stage responded to the shortcomings of its predecessor without replacing it, and the false-positive reduc-tion studies reviewed in this chapter position learned models as complements to static analysis rather than as substitutes for it [1]–[4].

SAST tools such as Semgrep, Bandit, and CodeQL examine source code without executing it and report findings by matching the code against curated rules or queries. Their behavior is deterministic and auditable, and they require no training data. Their weakness is equally well documented. A rule can only express patterns that its authors anticipated, so rule-based analysis both flags benign code as vulnerable and overlooks complex vulnerability patterns that match no rule. The resulting false-positive burden is substantial: an industrial study of static bug detection found that 328 of 433 alarms, approximately 76%, were false positives [3]. High false-positive rates erode developer trust and consume triage effort, which has motivated work that learns to re-rank or filter analyzer output [1]–[4].

Learning-based approaches treat vulnerability detection as a code classification problem. Pre-trained models such as CodeBERT [18], a bimodal model for program-ming and natural languages, and GraphCodeBERT [6], which incorporates data flow into pre-training, supply contextual representations that can be fine-tuned on labeled vulnerability data. Later work extends them with explicit data-flow embeddings [5], model ensembles [7], and multi-granular detection [8]. Such models can capture con-textual patterns that fixed rules cannot express, but their reported performance depends strongly on the data used to train and evaluate them. Widely used corpora such as Big-Vul, which consists of C/C++ functions, are built by mining vulnerability-fixing com-mits, a process shown to introduce label noise, duplication, and train–test leakage [10], [11]. When these defects were addressed, the F1-score of a state-of-the-art

7-billion-parameter code language model fell from 68.26% to 3.09% [11], which indicates that headline results on legacy benchmarks may overstate real-world capability. Because the present project analyzes Python source code, Python-native resources such as VU-DENC [12] and PyCode-Vul [13] are of particular interest.

Detection alone does not tell a developer why code is vulnerable or how to fix it. Retrieval-Augmented Generation (RAG) reduces the risk of ungrounded LLM out-put by supplying curated external knowledge at inference time; Vul-RAG, for example, retrieves distilled root-cause and fix-pattern knowledge to improve LLM-based vulner-ability judgments [14]. Automated repair has progressed from fine-tuned sequence-to-sequence models such as VulRepair [15] to LLM-generated patches, yet empirical analysis shows that such patches frequently fail security checks, functional checks, or both [16], and that iterative re-verification increases the share of patches that are both secure and correct [17]. Detection, explanation, repair, and verification have therefore each been studied, but largely in isolation. How they can be combined in a single work-flow is the question that frames this chapter.

The remainder of the chapter reviews the 17 numbered references of the project manuscript together with the original CodeBERT study [18]. Section 2.2 surveys them by theme, Section 2.3 compares them, Section 2.4 identifies the research gap, and Sec-tion 2.5 summarizes the findings and leads into the methodology.

2.2 Exhaustive Literature Survey

The surveyed studies are organized into five thematic groups. For each study, the objec-tive, method, principal findings, limitations, and relevance to the proposed framework are stated. Quantitative results are reported as published by the respective authors. Be-cause the studies use different datasets, data splits, and metrics, figures from different studies are not directly comparable and are not ranked here. Limitations described as inferences are the assessment of this report relative to the scope of the project and are not statements made by the cited authors. The sources also differ in maturity: ten appear in conference proceedings or journals [1], [3], [5], [6], [8], [10]–[12], [15], [18], six are arXiv preprints whose peer-review status is not recorded in the reference list [2], [4], [9], [14], [16], [17], one has no recorded venue [7], and one is a dataset [13]. The evidential weight of preprint findings is therefore treated as provisional.

2.2.1 Traditional Static Analysis and False-Positive Reduction

Rule-based analyzers remain the first line of automated defense, yet the literature repeat-edly identifies their precision as a barrier to adoption. Semgrep, Bandit, and CodeQL detect

vulnerabilities by matching code against rules or queries. The manuscript's ref-erence list contains no tool-specific publications for Semgrep or Bandit, so these tools are characterized here only by their role as rule-based detectors.

Kharkar et al. [1] aimed to reduce false positives in analytic bug detectors by learning from the detectors' own output. Their transformer-based re-ranker improved precision by approximately 17.5 percentage points on null-dereference and resource-leak bugs. The contribution is a demonstration that a learned model can sit downstream of a static analyzer and improve the quality of its results. The evaluation covers two bug categories, however, and its transfer to security vulnerability classes is not established by the material reviewed (inference). Xiong and Zhang [2] compared LLM-based coding agents for filtering false positives in vulnerability findings. They report that the false-positive rate fell from over 92% to as low as 6.3% on the OWASP Benchmark, and that up to 93.3% of false positives were correctly identified on real-world CodeQL alerts. These two figures come from different datasets and are expressed through different measures, a residual false-positive rate and the share of false positives identified, so they should not be read as a single performance level. The cost and latency of agent-based triage are not established in the material reviewed (inference).

Du et al. [3] moved the question into an industrial setting, studying LLM-based false-positive reduction on static-analysis alarms at Tencent. They found that 328 of 433 alarms, approximately 76%, were false positives, and that LLMs alone struggled with long code contexts. The study is valuable because it quantifies the false-positive problem outside benchmarks, but it describes a single organization, so its proportions should not be generalized without caution (inference). The limitation on code length is directly relevant to any design that feeds source code to a learned model. ZeroFalse [4] addresses precision by using an LLM as an adjudicator of static-analyzer findings enriched with Common Weakness Enumeration (CWE) information, rather than as a detector working from scratch. It reports F1-scores of 0.912 on the OWASP Java Benchmark and 0.955 on the OpenVuln dataset. Its principal limitation, in the context of this project, is scope: as summarized, it improves the triage of existing findings but does not produce developer-facing explanations or repairs (inference).

Taken together, these four studies establish that static analysis is useful but insuf-ficiently precise on its own, and that learned models can recover precision when placed downstream of the analyzer. They also share a structural limitation. A filter can only re-move findings that the analyzer has already reported, so it cannot recover vulnerabilities that the rules miss (inference). This observation supports retaining Semgrep and Bandit in the proposed framework for their

auditable rule coverage while adding an independent learned classifier that does not depend on a rule firing.

2.2.2 Transformer-Based Vulnerability Detection

CodeBERT [18] and GraphCodeBERT [6] are pre-trained code models that supply the foundation for most of the detection work reviewed here. CodeBERT is a bimodal model for programming and natural languages, whereas GraphCodeBERT incorporates data flow into the pre-training of its code representations. Neither is a vulnerability detector in itself. Each provides contextual representations that must be fine-tuned on labeled vulnerability data, which makes the quality of that data central to the outcome.

The DFEPT study [5] aimed to strengthen pre-trained backbones by augmenting them with explicit data-flow embeddings. It is reported to improve accuracy on De-vign and F1 on ReVeal, with values of 64.97% accuracy and 47.9% F1, respectively. The contribution is evidence that data-flow structure adds information beyond token se-quences. The modest absolute F1 on ReVeal also illustrates how difficult realistic bench-marks remain for transformer models. VulD-CodeBERT [7] combines CodeBERT with a BiLSTM and attention ensemble and reports 90.30% accuracy on C/C++ code. The model confirms that CodeBERT is a workable base for vulnerability detection, but it is restricted to C/C++, its venue is not recorded, and the ensemble adds architectural com-plexity relative to a single fine-tuned encoder (inference). StagedVulBERT [8], pub-lished in IEEE Transactions on Software Engineering, proposes multi-granular detec-tion with a novel pre-trained code model and reports 92.26% F1 at the function level together with statement-level localization. Localization is the most relevant contribu-tion for remediation, because it narrows the code that an explanation or a patch must address. The datasets and languages behind the reported figure are not established in the material reviewed, so its applicability to Python cannot be assumed.

Two observations follow. First, the reported values, for example 47.9% F1 for DFEPT on ReVeal and 92.26% F1 for StagedVulBERT, come from different datasets, languages, and protocols, and the differences may reflect data characteristics at least as much as model design (inference); Section 2.2.3 examines this concern. Second, all three detection studies rely on pre-trained code models, which supports fine-tuned CodeBERT as a reasonable and reproducible starting point for the project's classifier. It does not show that CodeBERT is the best available model, and the unified-dataset evidence discussed next favors encoder-only architectures as a family rather than any single member [9].

2.2.3 Vulnerability Datasets and Model Generalization

The reliability of every classifier discussed above depends on the dataset beneath it. BigVul, a widely used corpus of C/C++ functions, is assembled by mining vulnerability-fixing commits. Chen et al. [10] documented substantial label noise and duplication in such data when introducing DiverseVul as a partial remedy. Ding et al. [11] made the concern concrete by introducing PrimeVul, a benchmark designed to address Big-Vul's label inaccuracy, duplication, and data leakage through manual curation, stricter dedu-plication, and a chronological train–test split. Under an equivalent evaluation protocol, a state-of-the-art 7-billion-parameter code language model that scored 68.26% F1 on Big-Vul fell to 3.09% F1 on PrimeVul. The size of the drop is consistent with dupli-cated or leaked examples having inflated earlier results, although the summary available here does not attribute it to a single cause (inference). For the present purpose, the study's evidence concerns C/C++-oriented data, and Python coverage is not established (inference).

Safdar et al. [9] extended the critique across the wider landscape by unifying and cleaning Devign, Big-Vul, ReVeal, CrossVul, D2A, CVEfixes, DiverseVul, and PrimeVul into a single corpus of 236,663 samples. They found that encoder-only mod-els generalize best across datasets, reporting a 94.73% F1-score on Big-Vul and a 6.8 percentage-point absolute improvement in recall over CodeBERT. That Big-Vul figure appears high beside the PrimeVul result above, but the two come from different proto-cols and models, which reinforces the central lesson: a vulnerability-detection score is meaningful only together with the dataset, the split strategy, and the cleaning procedure that produced it. Taken together, the three studies suggest that dataset unification and careful model selection are more reliable routes to generalization than reliance on any single corpus (inference). The preprint status of the unified-corpus study also means that its conclusions remain provisional.

None of these benchmarks is described as Python-native in the material reviewed. VUDENC [12] is the closer fit. It pairs the before-and-after states of fix-ing commits to obtain a balanced Python dataset of 13,738 vulnerable and 13,738 non-vulnerable files, and reports F1-scores of 80–90% with an LSTM baseline. It demonstrates that a Python-focused learning setting is feasible, but its reported baseline is an LSTM and not a pre-trained transformer, and, because it also derives labels from commits, it may inherit the noise discussed above (inference). PyCode-Vul [13], a Python benchmark dataset hosted on Kaggle, is the dataset on which the present project trains and evaluates its classifier; after preprocessing it comprises 17,676 functions, which are expanded into 53,073 code chunks. It is not a peer-reviewed publication, and its labeling procedure, provenance,

and deduplication are not established by the material reviewed. These properties must therefore be treated as threats to validity. One related methodological detail deserves explicit documentation in the methodology chapter: when long functions are segmented into overlapping windows, splitting the data at the function level before segmentation avoids leakage between training and test partitions (inference).

2.2.4 Retrieval-Augmented Vulnerability Analysis

Vul-RAG [14] is the principal retrieval-augmented work in the reference list. Its ob-jective, as its title indicates, is to enhance LLM-based vulnerability detection through knowledge-level RAG. Rather than relying on the model alone, the method retrieves distilled root-cause and fix-pattern knowledge. The reported results are a 16–24 point improvement in pairwise accuracy over baseline LLMs and the discovery of ten pre-viously unknown vulnerabilities in the Linux kernel. The contribution is evidence that retrieved, abstracted vulnerability knowledge can improve LLM judgments beyond what the baseline models achieve. The study was evaluated on Linux kernel vulnerabilities, and its transfer to Python is not established. It is also a preprint, and it uses retrieval to support a detection judgment rather than to explain the output of a separately trained classifier (inference).

The project instead grounds retrieval in the Common Weakness Enumeration, a standardized catalogue of weakness types maintained by MITRE; the version used is 4.20. The numbered references include no study that evaluates CWE-grounded retrieval for explaining Python vulnerabilities. Indirect support exists: ZeroFalse [4] enriches analyzer findings with CWE information before LLM adjudication, which is consistent with CWE context being useful for LLM reasoning, though it does not isolate that ef-fect. The literature therefore supports retrieval grounding in principle but does not show that CWE-level knowledge improves the quality of explanations for Python code. For this reason, CWE-grounded RAG is positioned in the project as a means of support-ing explanations and remediation guidance, and not as a mechanism expected to raise classification accuracy.

2.2.5 Automated Vulnerability Repair and Verification

VulRepair [15] is a T5-based approach to automated vulnerability repair, fine-tuned with byte-pair encoding. It achieved a 44% Perfect Prediction rate, though accuracy fell sharply for repairs longer than ten tokens. The contribution is a demonstration that repair can be learned, and the length sensitivity shows that patch complexity is a first-order variable in repair reliability. In the summarized results, the study does not describe an independent verification stage; whether its

repairs remove the vulnerability under re-analysis is the question that the later studies take up (inference).

Al-Maamari [16] analyzed why LLM-generated security patches fail, using a tri-axis evaluation of compilation, security (through proof-of-vulnerability tests), and functional correctness. Of 319 patches for 64 real Java CVEs, only 24.8% were fully correct and 51.4% failed both the security and the functionality checks. Failures were dominated by semantic misunderstanding rather than syntax errors, so a patch that com-piles and looks plausible may still be wrong. The study is a preprint and concerns Java, but its message is general: patch correctness has several independent dimensions, and none can be assumed. Cheng [17] studied a Detect-Repair-Verify pipeline for LLM-generated code in a multi-language, multi-granularity setting. Iterative re-verification improved the yield of secure-and-correct code, particularly when the repair scope was bounded to the file or requirement level rather than the whole project. This is also a preprint on LLM-generated code, so its direct applicability to repairing detected vulnerabilities in existing Python code is an assumption (inference).

Together, these three studies indicate that automated repair is feasible but unreli-able, that failures of LLM-generated patches are dominated by semantic misunderstand-ing [16], and that re-verification with bounded scope improves outcomes [17]. They provide the empirical justification for treating generated patches as suggestions that must be re-analyzed, and for limiting repair to a bounded unit of code instead of attempting project-wide changes.

2.3 Comparative Analysis of Existing Research

The reviewed approaches differ first in detection methodology. The false-positive reduction studies [1]–[4] operate downstream of a static analyzer, using a transformer re-ranker, LLM agents, or an LLM adjudicator. The transformer-based detectors [5]–[8], [18], as summarized, classify code directly and do not depend on analyzer output. Vul-RAG [14] occupies a third position, using retrieved knowledge to guide an LLM's judgment. The two families have complementary profiles. Filtering approaches improve precision but are bounded by what the analyzer reports, whereas direct classifiers can flag code that no rule matches but must manage their own false positives. This asym-metry is the principal technical argument for combining the two.

Programming-language coverage is uneven. Evidence is reported on the OWASP Java Benchmark [4], on real Java CVEs [16], on C/C++ code [7], and on Linux kernel vulnerabilities [14], while one study spans several languages [17]. In the material re-viewed, only VUDENC [12] and

PyCode-Vul [13] are Python resources, and both are datasets rather than end-to-end tools. For the remaining studies, language coverage is either non-Python or not established, so their conclusions cannot be assumed to hold for Python.

Dataset characteristics are a cross-cutting concern. Label noise, duplication, and leakage [10], [11] mean that many reported detection scores are not comparable across studies, which is why this chapter reports figures without ranking them. False positives dominate the SAST literature [1]–[4], whereas missed vulnerabilities (false negatives) surface mainly through the generalization failures of learned detectors [9], [11]. Because leakage can also inflate apparent recall, it may conceal such misses (inference).

Explainability and remediation support is the dimension on which the reviewed set is thinnest. Within it, statement-level localization [8], knowledge-level retrieval [14], and CWE-enriched adjudication [4] each contribute part of what a developer needs, and only three studies address repair [15]–[17]. Patch verification is addressed explicitly by two studies, one measuring patch correctness [16] and one incorporating re-verification into a pipeline [17].

The false-positive studies address static analysis and learned triage but, as summarized, not explanation or repair. The detection and dataset studies address classification and its evaluation but not remediation. The repair studies address patch generation and verifi-cation, and the Detect-Repair-Verify pipeline [17] comes closest to an end-to-end loop, yet it is described as a study of LLM-generated code and does not, as summarized, pair a classifier trained on Python data with retrieval-grounded explanation. This separation, rather than any deficiency in the individual studies, is what the next section builds on.

2.4 Research Gap Identification

The preceding review shows that each capability required of a practical security assistant has been studied, but in separate lines of work. Static analysis with false-positive reduc-tion, transformer-based classification, retrieval-augmented analysis, and LLM-based re-pair with verification are each supported by several studies. The research gap addressed by the project follows from this separation: no reviewed work integrates static analysis, machine-learning classification, retrieval-augmented explanation, and automated patch verification within a single evaluated pipeline for Python source code. Three features of the literature make this a substantive problem and not merely a matter of incomplete coverage. First, the strongest reported results are tied to particular languages and bench-marks that are mostly not Python. Second, the evidence on dataset quality shows that component-level results cannot be assumed to transfer to a new setting. Third, the docu-

mented failures of LLM-generated patches show that a pipeline that stops at generation leaves the developer with unverified output.

The first two components follow from the false-positive and detection literature. Semgrep and Bandit provide deterministic, auditable findings, but the false-positive bur-den [1]–[4] and the inability of filters to recover unreported vulnerabilities argue against relying on them alone. A fine-tuned CodeBERT classifier supplies a contextual judg-ment that does not depend on a rule firing [5]–[8], [18], and encoder-only models are reported to generalize well [9]. The dataset literature [10], [11] adds the condition that evaluation must avoid leakage, and Python-native resources [12], [13] make training on Python code feasible. The two components are combined because their strengths are complementary, not because the combination is assumed to be superior. Whether adding static-analysis findings to the classifier's judgment improves overall performance is an empirical question for the evaluation chapters.

The third and fourth components respond to the observation that a detection re-sult is not directly actionable. Knowledge-level retrieval improves LLM judgments [14], and CWE information has been used to enrich analyzer findings [4]. Grounding expla-nations in a standardized weakness catalogue gives a developer a traceable source for the reasoning, which an unconstrained LLM explanation does not provide. AI-assisted remediation is supported by evidence that repair can be learned [15], but the failure anal-ysis of generated patches [16] argues for presenting the output as guidance and not as an authoritative correction.

Post-repair verification is the component with the clearest empirical motivation and the thinnest Python evidence. Most LLM-generated patches fail at least one correct-ness criterion [16], and iterative re-verification increases the yield of secure-and-correct code, particularly when the repair scope is bounded [17]. A re-analysis step turns patch generation from an unchecked output into a hypothesis that is tested. Its limits must also be stated. Re-analysis can show that a patched fragment no longer triggers the same detectors, but it cannot by itself show that the intended functionality is preserved, which the failure analysis treats as a separate dimension [16]. Re-analysis is therefore a neces-sary condition for a trustworthy patch, not a sufficient one, and its result is reported as an outcome and not as a guarantee.

The proposed AI Security Copilot addresses the integration gap by connecting these components in one workflow. Source code is preprocessed and analyzed with Semgrep and Bandit; a fine-tuned CodeBERT model classifies the code; CWE-grounded retrieval supplies context for an AI-generated explanation and patch suggestion; the sug-gested patch is re-analyzed; and the results

are presented in a dashboard. Throughout, the classifier's binary prediction is distinguished from the wider workflow: the classifier is one component, and the effectiveness of the end-to-end system is a separate question. The contribution claimed is the integration of these capabilities for Python code, and not superiority over existing methods. The reviewed literature gives no basis for claiming that the integrated system outperforms other approaches, and generated patches are not assumed to be secure. Limitations carried forward into the later chapters include theunconfirmed provenance of PyCode-Vul, the dependence of the explanation component on the quality of the retrieved CWE content and of the underlying language model, and the restriction of verification to re-analysis (inference).

2.5 Chapter Summary

This chapter reviewed 18 studies covering static analysis and false-positive reduction, transformer-based vulnerability detection, vulnerability datasets, retrieval-augmented analysis, and automated repair with verification. Four conclusions emerge. First, rule-based static analysis is valuable but imprecise on its own, learned triage improves its pre-cision [1]–[4], and such triage cannot recover what the rules miss. Second, pre-trained transformer encoders form the dominant basis for learned detection [5]–[8], [18], but re-ported performance is conditioned on dataset quality [9]–[11], so the evaluation protocol matters as much as the architecture, and Python-native resources [12], [13] are scarce relative to C/C++ ones. Third, retrieval of vulnerability knowledge improves LLM rea-soning [14], although CWE-grounded retrieval for explaining Python vulnerabilities has not been directly evaluated in the reviewed set. Fourth, LLM-generated patches are unre-liable without independent verification, and re-verification with bounded scope improves outcomes [15]–[17].

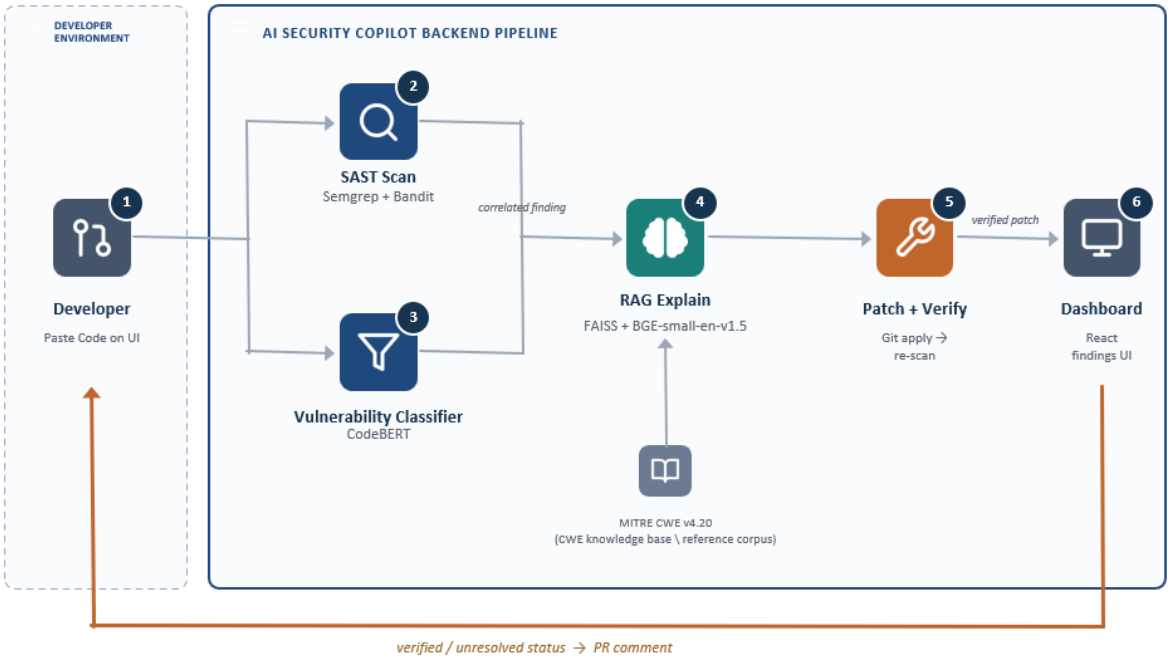
These findings define the research gap: no reviewed work combines rule-based analysis, contextual classification, CWE-grounded retrieval, remediation guidance, and post-repair verification in a single Python-focused pipeline. They also set out the design requirements for the next chapter. Rule-based analysis is retained, CodeBERT is fine-tuned on Python data with leakage-aware evaluation, explanations are grounded in CWE knowledge, patches are treated as suggestions subject to re-analysis, and the verification outcome is reported. Chapter 3 presents the methodology and implementation of the AI Security Copilot that follows from these requirements.

Chapter 3

Methodology and Implementation

This chapter describes the methodology and implementation of the proposed AI-Assisted Multi-Stage Framework for Source Code Vulnerability Detection. The system combines Static Application Security Testing (SAST), deep-learning-based vulnerability classification, and Retrieval-Augmented Generation (RAG) to support the identification and analysis of software vulnerabilities. Semgrep and Bandit are used to identify potentially insecure coding patterns, while a fine-tuned CodeBERT model classifies source-code functions as vulnerable or non-vulnerable. The retrieval component uses BGE embeddings and FAISS to identify relevant security information from the MITRE Common Weakness Enumeration (CWE) knowledge base. The chapter explains the system architecture, dataset preparation, model configuration, software components, retrieval methodology, and the overall workflow for vulnerability detection and remediation support.

3.1 Block diagram



3.2 Hardware description

This section describes the hardware required to develop and execute your project. Since your system uses static analysis, a transformer-based machine learning model, RAG, and an LLM, explain the hardware in terms of development, inference, and application execution.

3.2.1 Hardware Configuration

Component	Description
Processor	I5
RAM	16 GB
Storage	16 GB
Operating System	Windows

3.2.2 Hardware Requirements

The proposed AI-Assisted Software Vulnerability Detection system requires adequate processing power and memory to perform static code analysis, machine learning-based vulnerability classification, embedding-based information retrieval, and web application execution. The hardware requirements depend on the size of the source code, model configuration, input sequence length, and the number of tasks executed simultaneously.

Semgrep and Bandit primarily utilize CPU resources to analyze source code and identify potential security vulnerabilities. The fine-tuned CodeBERT model performs inference to classify code as vulnerable or non-vulnerable and can operate using either a CPU or a compatible GPU. However, GPU acceleration may improve inference speed for larger workloads. Sufficient RAM is required to load the model, process source code, and execute the backend and frontend components efficiently.

The RAG component requires memory and processing resources to generate embeddings and retrieve relevant cybersecurity information from the vector database. The React frontend and FastAPI backend also require resources for application execution and communication between components. If an externally hosted Large Language Model (LLM) is used, its text-generation workload is handled by the remote service, reducing the local computational requirements for generation.

3.3 Software description, flowchart / algorithm

3.3.1 Software Requirements and Technologies

Software / Technology	Purpose
Python	Backend development and machine learning integration
FastAPI	Backend API and orchestration of system components
React	User interface for code submission and displaying findings
Semgrep	Rule-based static security analysis
Bandit	Detection of common Python security issues

CodeBERT	ML-based vulnerable/non-vulnerable code classification
BGE embeddings	Converts security documents into vector representations
FAISS	Similarity-based retrieval of relevant security knowledge
CWE and OWASP	Security knowledge sources for vulnerability context
LLM	Generates vulnerability explanations and remediation suggestions
Git	Version control and, if implemented, patch tracking

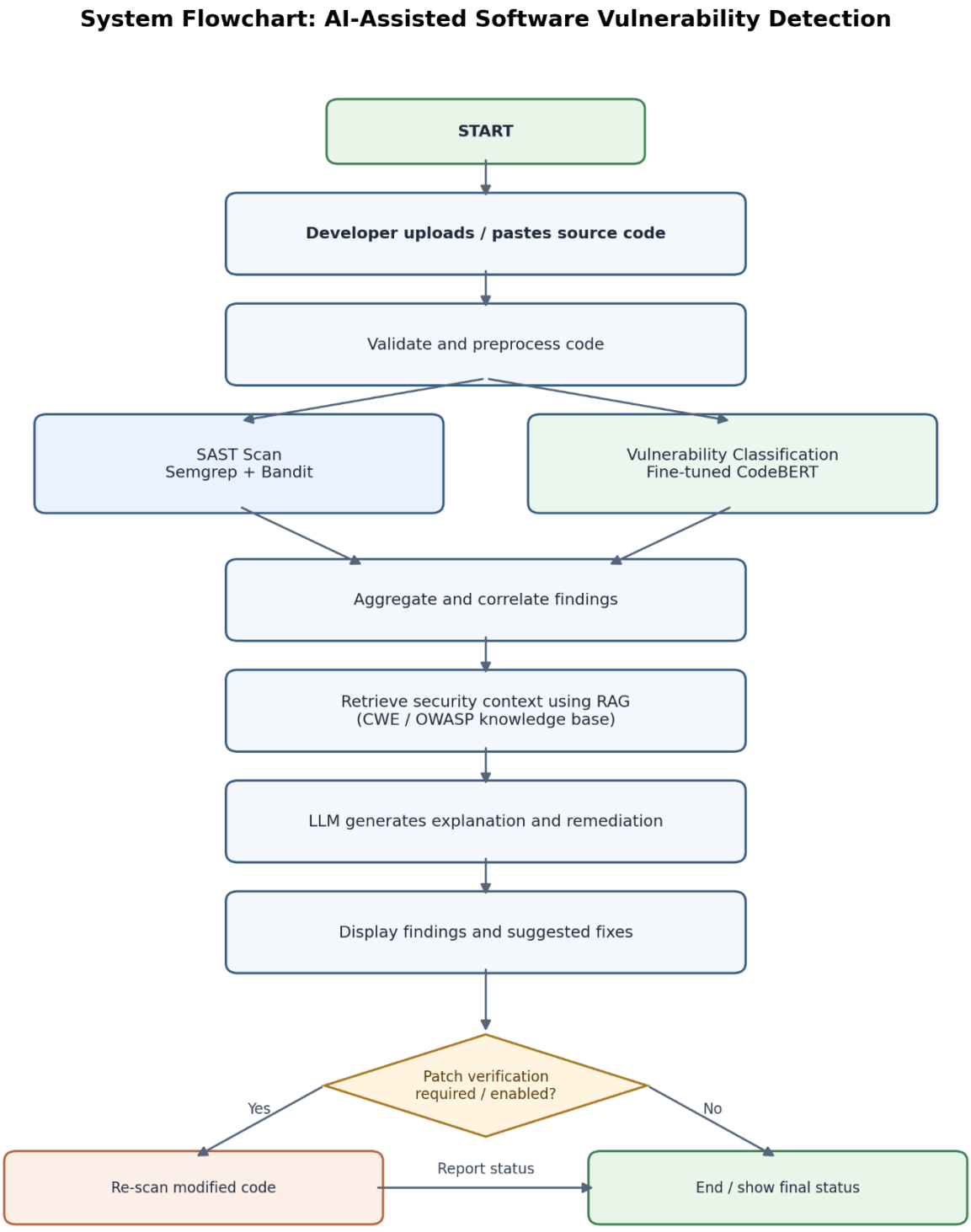
3.3.2 System Implementation

The system is implemented as a modular software application that integrates static security analysis, machine learning-based vulnerability classification, retrieval-augmented generation, and AI-assisted remediation. The process begins when a developer submits source code through the React-based user interface. The FastAPI backend receives the input, validates it, and coordinates the analysis pipeline.

The source code is analyzed using Semgrep and Bandit to identify potentially insecure coding patterns and common Python security issues. In parallel, the fine-tuned CodeBERT model classifies the code as potentially vulnerable or non-vulnerable and produces a classification score. The outputs are subsequently consolidated to provide a unified view of the detected issues.

The RAG module retrieves relevant security information from the indexed CWE and OWASP knowledge sources using BGE embeddings and FAISS. The retrieved information, along with the detection evidence, is supplied to the LLM to generate vulnerability explanations and potential remediation recommendations. The results are presented through the web interface, allowing developers to review findings and suggested fixes. Where patch verification is implemented, the modified code is re-scanned to assess whether the original findings persist.

3.3.3 System Flowchart



3.3.5 Actual Implementation Screenshots

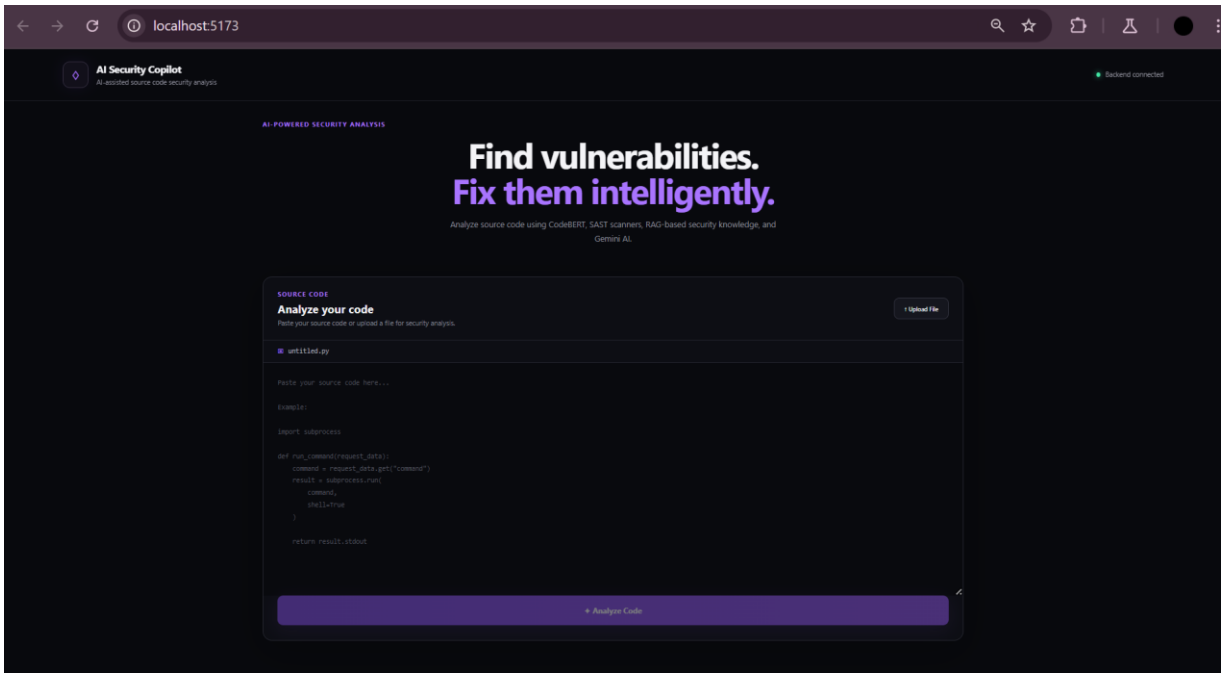


Figure 1 Source Code Submission for Developers

The screenshot displays the web interface through which developers submit or paste source code for security analysis. The interface provides a code-entry area and an analysis control to initiate the vulnerability detection process. Once the request is submitted, the backend processes the source code through the configured analysis pipeline.

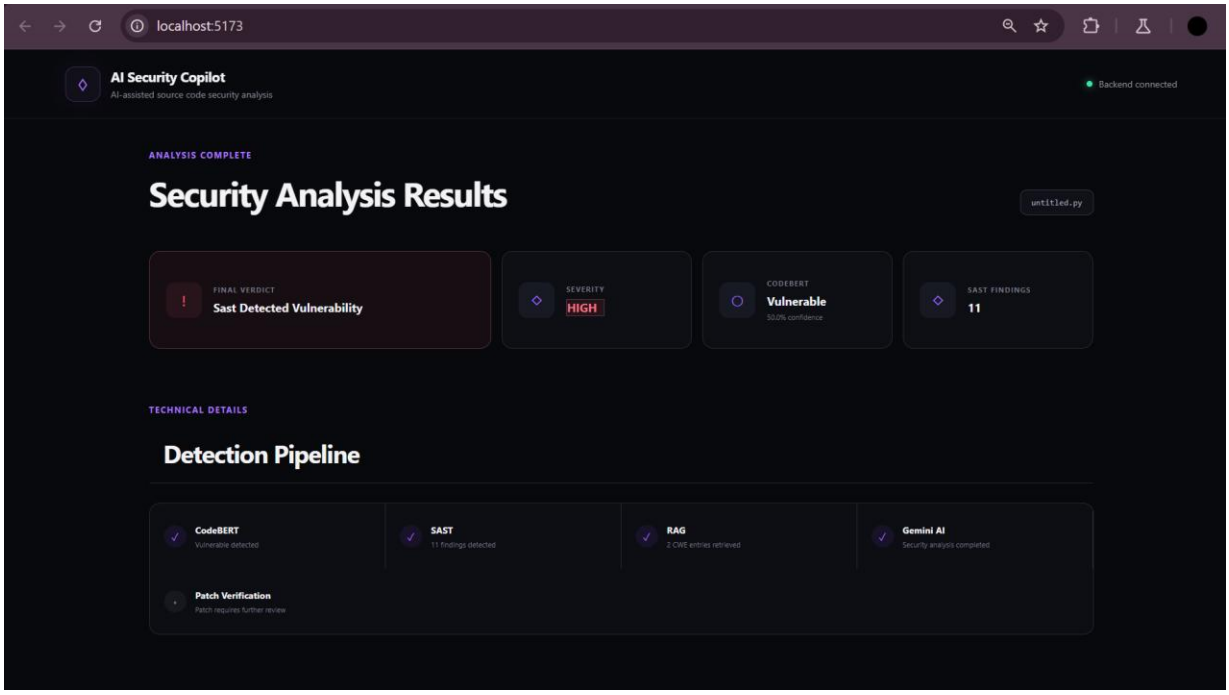


Figure 2 Security Analysis Results Dashboard

The screenshot shows the results dashboard generated after the source code has been analyzed. It presents an overview of the detected security issues and summarizes the status of the detection pipeline. The dashboard helps developers review the analysis outcome and understand which stages of the vulnerability detection workflow have produced results.

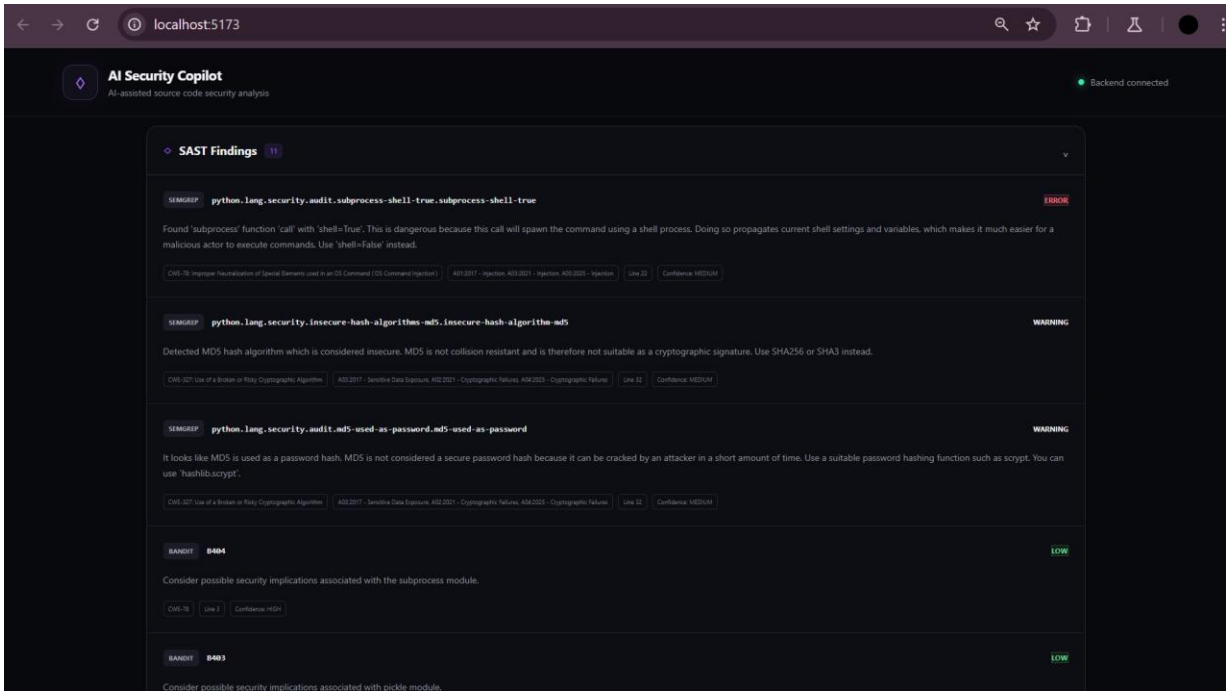


Figure 3 Static Application Security Testing (SAST) Findings

The screenshot presents the findings produced by the static analysis component of the system. The interface displays potential security issues identified through source-code scanning,

allowing developers to review reported vulnerabilities and their associated details. These findings help identify insecure coding patterns that may require further investigation and remediation.

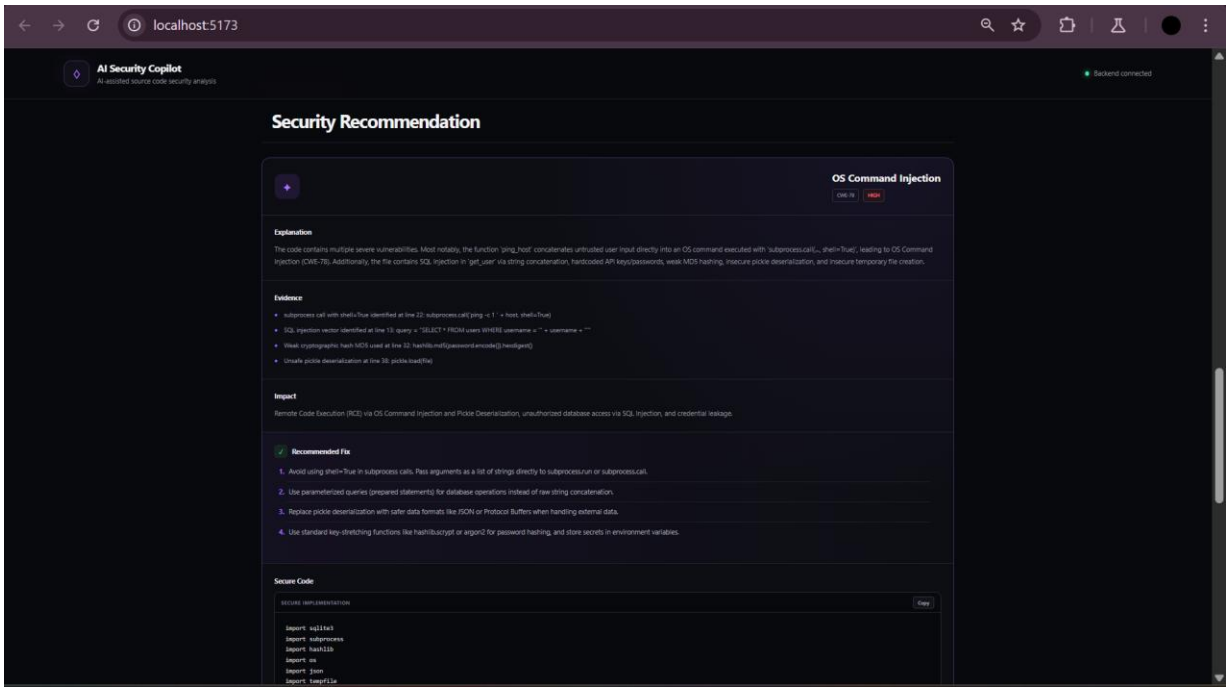


Figure 4 AI-Generated Security Recommendations and Remediation Guidance

The screenshot illustrates the security recommendation interface, which presents AI-assisted explanations and potential remediation strategies for identified vulnerabilities. The recommendations aim to help developers understand the underlying security weaknesses and identify possible corrective actions. Suggested changes should be reviewed and validated before they are incorporated into the application.

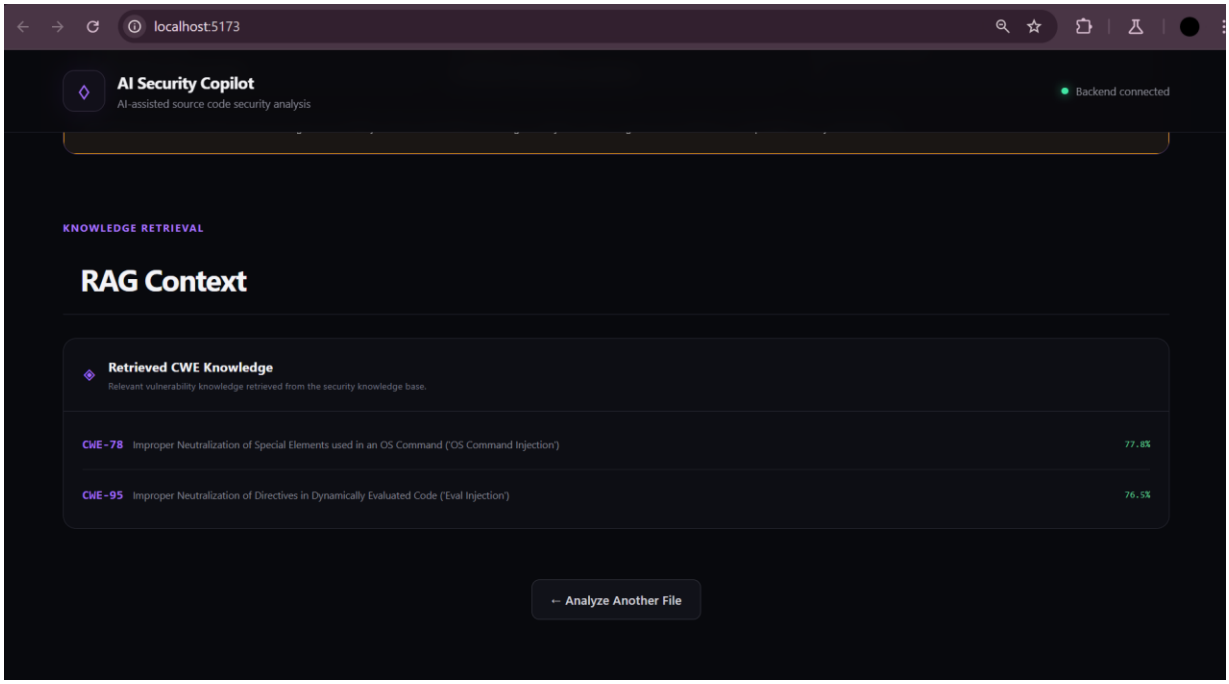


Figure 5 Retrieved Cybersecurity Knowledge Using Retrieval-Augmented Generation (RAG)

The screenshot shows the RAG context section of the application, where relevant cybersecurity knowledge is presented to support vulnerability analysis. The retrieved information provides contextual references that the language model can use when explaining security weaknesses and suggesting remediation strategies. This approach helps ground AI-generated responses in relevant security documentation rather than relying solely on the model's internal knowledge.

3.4 Chapter Summary

This chapter presented the methodology and implementation approach of the proposed source-code vulnerability detection framework. The system integrates static analysis using Semgrep and Bandit, CodeBERT-based vulnerability classification, and semantic retrieval using BGE embeddings and FAISS over the CWE knowledge base. The PyCode-Vul dataset is used for classifier development and evaluation, with preprocessing techniques applied to improve data quality. The methodology establishes a multi-stage workflow that connects vulnerability detection with relevant security knowledge and supports a remediation process involving patch application and re-scanning. These components provide the foundation for evaluating the classifier and retrieval system, with their results and analysis discussed in the following chapter.

Chapter 4
Results and Analysis

4.1 Introduction

This chapter reports the available evaluation evidence for the AI-Assisted Multi-Stage Framework for Source Code Vulnerability Detection. It covers the PyCode-Vul dataset and preprocessing, the role of Static Application Security Testing (SAST) through Semgrep and Bandit, CodeBERT classification results, and the retrieval evaluation for the BGE–FAISS component. The evidence is reported by component because the available results do not establish that the complete explanation, patch-generation, user-interface, and verification workflow has been quantitatively evaluated end to end.

4.2 Dataset and Evaluation Setup

The project uses the PyCode-Vul function-level dataset as the primary source of labelled Python code for training and evaluating the vulnerability classifier. Samples are labelled as vulnerable or non-vulnerable. The project documentation describes associated metadata such as vulnerability identifiers, CWE/CVE information, repository details, file paths, and commit information as being retained for inspection and auditing rather than supplied as model input. The classifier receives source-code content and predicts the vulnerability class.

Preprocessing included inspection of missing values and duplicates, removal of samples with missing or empty source code, source-code normalisation for duplicate detection, and overlap checks across dataset partitions to reduce data leakage. The data was split into training, validation, and test partitions using stratification to preserve class proportions. CodeBERT input was prepared in token windows of up to 512 tokens with a 128-token stride for longer functions. The project description also specifies a maximum of 16 chunks per function and mean-probability aggregation of chunk-level predictions to obtain a function-level prediction.

Table 4.1: PyCode-Vul dataset after preprocessing and chunk generation

Split	Functions	Non-vulnerable	Vulnerable	Code chunks
Training	12,025	6,742	5,283	36,301
Validation	2,123	1,190	933	6,220
Test	3,563	1,962	1,601	—
Total	17,711	9,894	7,817	—

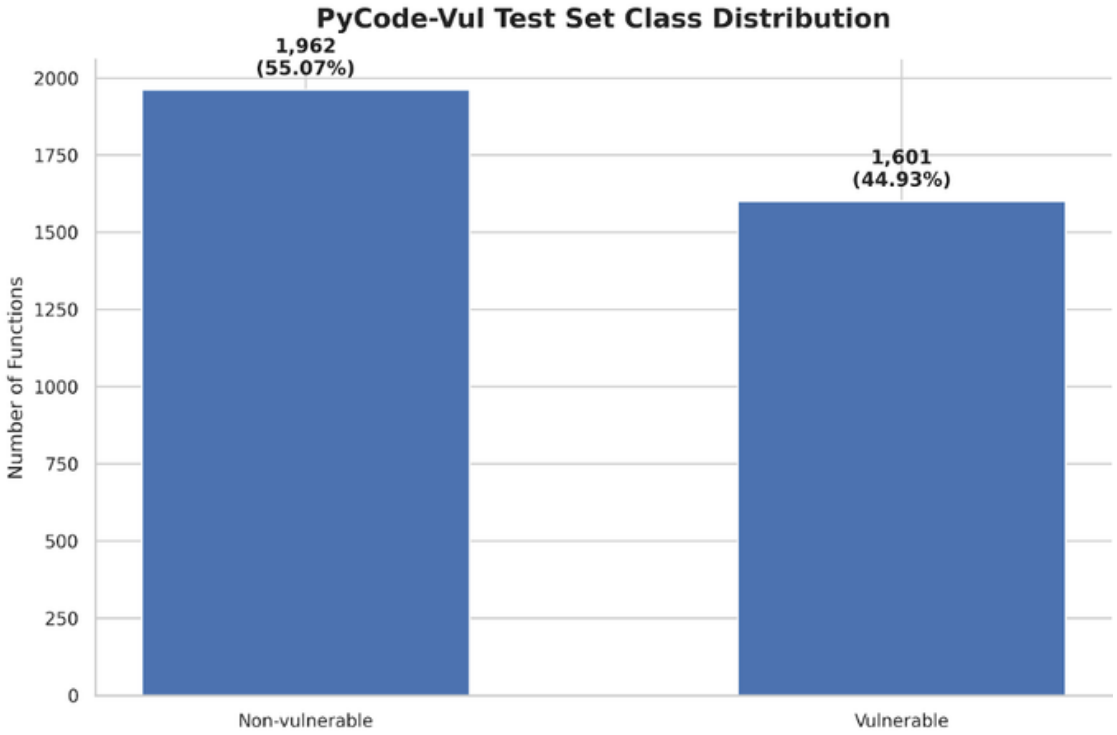


Figure 4.1: Class distribution shown for the CodeBERT test-set evaluation.

The held-out test set used for the reported evaluation contains 3,563 functions: 1,962 non-vulnerable and 1,601 vulnerable. Figure 4.1 presents this class distribution, while Section 4.4 summarises the classifier predictions.

4.3 Static Application Security Testing (SAST)

The SAST layer uses Semgrep and Bandit to inspect source code for patterns associated with potential security weaknesses. Semgrep applies configurable pattern-based rules, while Bandit focuses on common security issues in Python source code. These tools provide rule-based findings that complement the learned CodeBERT classifier; they are not themselves deep-learning models.

In the proposed workflow, source code is first scanned by Semgrep and Bandit to produce candidate findings. CodeBERT independently classifies functions as vulnerable or non-vulnerable. The architecture then calls for findings from the analysis components to be correlated, so that the system can consolidate scanner alerts and model predictions before retrieving relevant CWE security context. The SAST results can therefore help identify suspicious code patterns, while the classifier offers a separate learned assessment.

The materials available for this chapter do not provide a verified table of SAST true positives, false positives, scan coverage, execution time, or per-tool detection rates. Accordingly, Semgrep and Bandit are discussed as implemented/integrated analysis

components in the documented workflow, but no quantitative SAST effectiveness claim is made here. If scan logs are available, a later version of this section can report counts of findings by tool and rule, deduplicated findings, and manually verified outcomes.

Table 4.2: Role of the analysis components

Component	Method	Purpose in the framework
Semgrep	Rule/pattern-based static analysis	Flags source-code patterns that may indicate security weaknesses.
Bandit	Python-focused static analysis	Flags common security-relevant patterns in Python code.
CodeBERT	Fine-tuned transformer classifier	Predicts whether a code function is vulnerable or non-vulnerable.
BGE + FAISS	Embedding-based retrieval	Retrieves relevant CWE passages to support security context.

4.4 CodeBERT Classifier and Metrics

The classifier is based on microsoft/codebert-base and is fine-tuned for binary sequence classification. CodeBERT is a transformer model pre-trained on natural language and programming-language data. For this project, it predicts a vulnerable or non-vulnerable label for source-code functions. Longer functions are divided into overlapping chunks because the model accepts at most 512 tokens per input. The project documentation describes a 128-token stride, a maximum of 16 chunks per function, mean-probability aggregation, and a classification threshold of 0.5.

The classifier metrics in Table 4.3 are calculated from the 3,563-example confusion matrix, using vulnerable code as the positive class.

Table 4.3: Reported CodeBERT classification metrics

Metric	Reported value	Meaning
Accuracy	96.88%	Share of test examples classified correctly.
Precision	95.76%	Share of predicted vulnerable examples that are labelled vulnerable.
Recall	97.38%	Share of actual vulnerable examples identified by the classifier.
F1-score	96.56%	Harmonic mean of precision and recall.
ROC–AUC	99.32%	Reported area under the receiver operating characteristic curve.

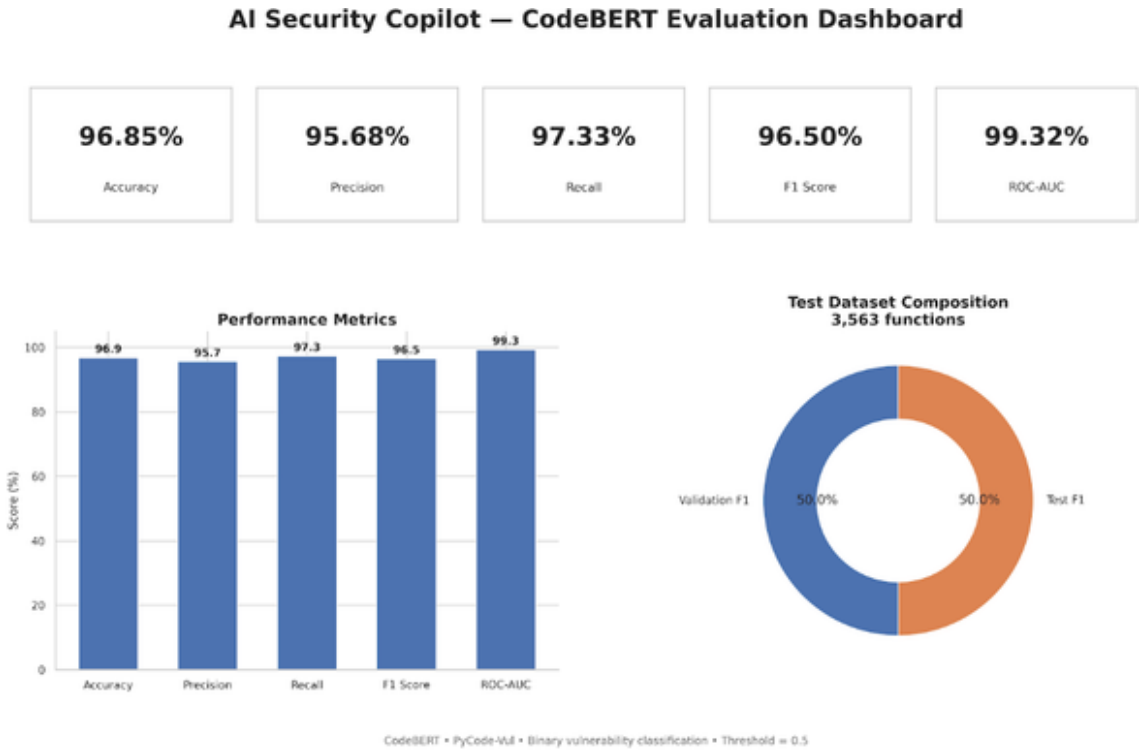


Figure 4.2: CodeBERT evaluation dashboard from the project materials.

The reported recall is higher than precision, indicating that the evaluation reports identification of a high proportion of vulnerable functions, while some non-vulnerable functions are still flagged. High classification scores are promising for the classifier component, but they do not demonstrate that every vulnerability type will be detected or that explanations and remediation suggestions are correct.

4.5 Confusion Matrix and Error Analysis

The confusion matrix in the project materials treats vulnerable code as the positive class. It reports 1,559 true positives (vulnerable functions correctly identified), 42 false negatives (vulnerable functions classified as non-vulnerable), 69 false positives (non-vulnerable functions flagged as vulnerable), and 1,893 true negatives (non-vulnerable functions correctly classified). These counts sum to 3,563 examples.

Table 4.4: Confusion matrix counts as displayed in the project evaluation

Actual class	Predicted vulnerable	Predicted non-vulnerable	Total
Vulnerable	1,559 (TP)	42 (FN)	1,601
Non-vulnerable	69 (FP)	1,893 (TN)	1,962
Total	1,628	1,935	3,563

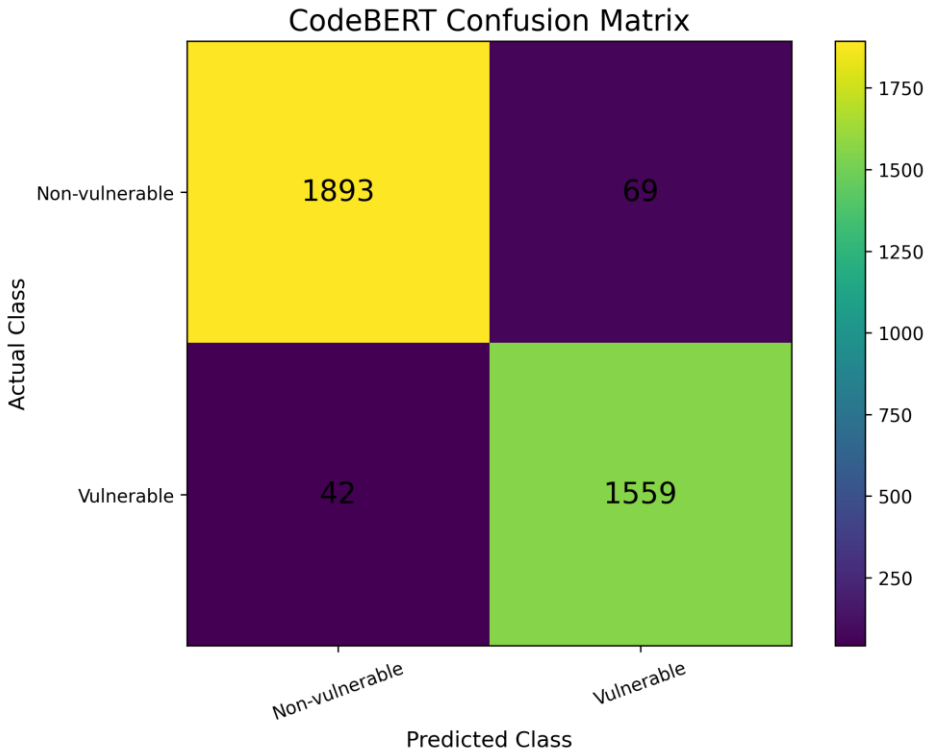


Figure 4.3: CodeBERT confusion matrix for the displayed evaluation run.

False negatives are particularly important in a security context because a vulnerable function predicted as safe may not receive additional review. The 42 false negatives should be inspected to determine whether they share particular vulnerability patterns, code structures, or labelling issues. False positives can increase triage effort and may reduce developer confidence if they occur frequently; the 69 false positives should therefore also be examined. Neither error type should be considered negligible in a security-sensitive workflow.

The confusion matrix shows that the classifier correctly identifies most vulnerable and non-vulnerable functions. The remaining false positives and false negatives provide clear targets for further error analysis and model refinement.

4.6 Retrieval-Augmented Generation (RAG) Retrieval Evaluation

The retrieval component uses the BAAI/bge-small-en-v1.5 embedding model to represent queries and security-document passages as vectors. FAISS is used to search the vector index. The recorded evaluation output states that the index contains 6,928 vectors and that retrieval was evaluated using 47 queries. The knowledge source is described in the project materials as the MITRE CWE v4.20 reference corpus. Retrieval is intended to provide relevant weakness descriptions, detection guidance, examples, consequences, and mitigation information to support later explanations.

Table 4.5: Recorded RAG retrieval evaluation metrics (47 queries)

Metric	Recorded value	Interpretation
Recall@1	1.0000	Expected relevant target was retrieved within the first result for the evaluation queries.
Recall@3	1.0000	Expected relevant target was present within the top three results.
Recall@5	1.0000	Expected relevant target was present within the top five results.
Precision@5	0.8000	Average relevance proportion among the top five retrieved results, as defined by the evaluator.
MRR@5	1.0000	Mean reciprocal rank of the expected result within the top five.
Section recall@5	1.0000	Expected CWE section was retrieved within the top five.
Top-1 CWE accuracy	1.0000	Top-ranked CWE matched the expected CWE in the recorded evaluation.
Top-1 section accuracy	0.8936	Top-ranked section matched the expected section for 89.36% of queries.

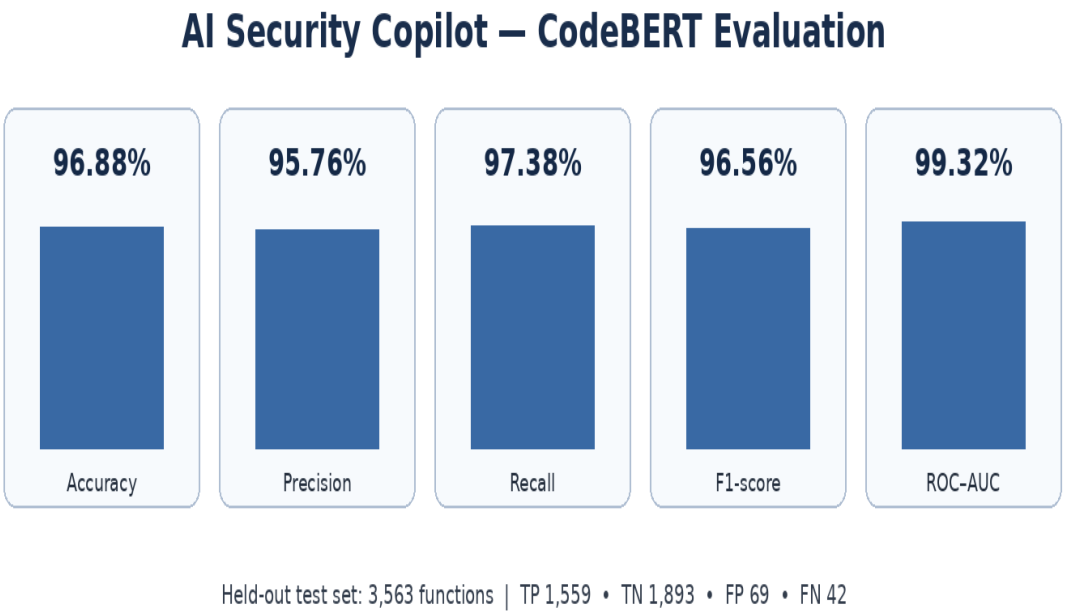


Figure 4.4: Recorded RAG retrieval metrics for the 47-query evaluation.

The recorded results indicate that the expected CWE target was retrieved at rank one for the evaluation set, while top-1 section accuracy was lower at 89.36%. Precision@5 of 0.80 indicates that not every item among the five retrieved passages was considered relevant under the evaluator's criteria. These measures assess retrieval behaviour only. They do not measure whether a generated explanation is factually correct, whether a proposed patch fixes the vulnerability, or whether the complete system prevents regressions.

Because the retrieval scores are very high on a relatively small set of 47 queries, the evaluation should be expanded with more varied queries, additional CWE categories,

ambiguous questions, and cases where the relevant passage is not easy to retrieve. The evaluation query set and relevance labels should also be reviewed to ensure that they fairly represent intended use.

4.7 Combined Interpretation of the Framework

The results show separate evidence for two components: CodeBERT classification and CWE knowledge retrieval. The SAST layer provides rule-based candidate findings, the classifier predicts function-level vulnerability labels, and the retrieval component searches for relevant CWE context. Together, these components are intended to support a workflow that moves beyond a scanner-only list of alerts by linking code findings with learned classification and security-reference information.

The available results do not provide a single end-to-end score for the complete framework. In particular, the classifier metrics do not measure SAST precision or recall, retrieval scores do not validate generated explanations, and neither set of metrics demonstrates patch correctness. The project materials identify full LLM integration, user-interface integration, and end-to-end patch validation as remaining work. Therefore, the most defensible conclusion is that the classifier and retrieval components show promising component-level results, while the complete workflow requires additional integration and validation.

4.8 Validity Considerations

The reported results depend on the composition and labelling of PyCode-Vul and may not generalise to all repositories, languages, coding styles, or previously unseen vulnerability types. Duplicate removal, source normalisation, stratified splitting, and overlap checks help reduce the risk of data leakage, but the checks should be documented and reproducible. The conflict between the 3,528-function split table and the 3,563-example confusion matrix must be resolved before the numbers are presented as a single experiment.

The RAG evaluation is based on 47 queries and uses target CWE/section expectations. Its results should be interpreted within that scope. The project materials do not provide verified quantitative SAST results or an end-to-end evaluation protocol for patch safety. No compliance with a particular testing standard is claimed in this chapter because the available material does not identify a specific standard used.

4.9 Chapter Summary

This chapter described the PyCode-Vul dataset and preprocessing, the complementary role of Semgrep and Bandit SAST, CodeBERT classification results, and BGE–FAISS retrieval evaluation. The project materials report high CodeBERT classification metrics and strong retrieval scores over 47 queries. However, the dataset totals and classifier metrics contain inconsistencies that must be checked against the final logs. Additional quantitative evaluation is needed for SAST, explanation quality, patch success, regression risk, and the integrated end-to-end workflow.

Chapter 5
Advantages, Limitations and Applications

5.1 Introduction

This chapter discusses the advantages, limitations, and potential applications of the proposed AI-Assisted Multi-Stage Framework for Source Code Vulnerability Detection. The framework combines Static Application Security Testing (SAST), deep-learning-based vulnerability classification, and Retrieval-Augmented Generation (RAG) to improve the identification and understanding of software vulnerabilities. Its integration of Semgrep, Bandit, CodeBERT, and CWE-based knowledge retrieval provides a structured approach to security analysis and remediation support. This chapter highlights the key benefits of the proposed system, examines the challenges associated with its current implementation, and explores its potential applications in secure software development.

5.2 Advantages

1. Multi-Layered Vulnerability Detection: The proposed system combines rule-based static application security testing using Semgrep and Bandit with machine-learning-based vulnerability classification using CodeBERT. This multi-layered approach provides complementary detection mechanisms and helps identify a wider range of potentially vulnerable code patterns.

2. Context-Aware Code Analysis: The fine-tuned CodeBERT model analyzes Python source code at the function level to identify potentially vulnerable patterns. This enables the system to detect suspicious code structures that may not be explicitly covered by predefined static analysis rules.

3. Automated Security Scanning: The system automates the initial security analysis of submitted source code, reducing the need for developers to manually inspect every function for potential vulnerabilities. It helps streamline the preliminary security review process.

4. Detailed Vulnerability Explanations: The Retrieval-Augmented Generation (RAG) component retrieves relevant security information from knowledge resources such as the CWE Top 25 and OWASP Top 10. This information supports the generation of explanations that help developers understand the nature and potential impact of identified vulnerabilities.

5. Remediation Assistance: The system provides suggestions for correcting insecure code and addressing identified weaknesses. These recommendations help developers understand possible corrective measures and encourage the adoption of more secure coding practices.

6. Centralized Web Interface: The system integrates its vulnerability detection and explanation components into a centralized web interface. Developers can submit source code, review detected findings, and access explanations and remediation suggestions through a single platform.

7. Reduced Manual Effort: By automating parts of vulnerability identification, analysis, and reporting, the system can reduce the time and effort required for initial security reviews. This allows developers to focus their attention on investigating significant findings and implementing appropriate corrections.

8. Local Deployment Capability: The system can be deployed on a local machine with sufficient computational resources, allowing source-code analysis to be performed locally. However, when an externally hosted language model API is used for vulnerability explanations or remediation, the relevant code or contextual information sent to that service may leave the local environment.

5.3 Limitations

1. Possibility of False Positives and False Negatives: Semgrep, Bandit, and the CodeBERT classifier may produce false positives by flagging safe code as vulnerable or false negatives by failing to identify actual vulnerabilities. Therefore, the findings require review and validation before corrective action is taken.

2. Dependence on Training Data: The performance of the fine-tuned CodeBERT model depends on the quality, diversity, and representativeness of its training dataset. Its effectiveness may decrease when analyzing code patterns, vulnerability types, or programming practices that are not adequately represented in the training data.

3. Programming Language Limitation: The fine-tuned CodeBERT classifier is designed for Python vulnerability detection. Its predictions have not been validated for other programming languages, which limits the applicability of the machine-learning component to projects written in different languages.

4. Limited Contextual Understanding: The system primarily analyzes source code at the function level. Consequently, it may fail to identify vulnerabilities that depend on interactions

between multiple functions, application components, external dependencies, configuration files, or runtime execution conditions.

5. Incomplete Vulnerability Coverage: The system cannot guarantee the detection of every security weakness. Complex business-logic vulnerabilities, runtime issues, certain configuration weaknesses, and vulnerabilities requiring dynamic execution or deeper application-level analysis may remain undetected.

6. Dependence on Retrieved Knowledge: The quality of RAG-based explanations depends on the relevance, accuracy, and completeness of the retrieved security documentation. If the retrieval process returns insufficient or unrelated information, the resulting explanations and remediation recommendations may be incomplete or inaccurate.

7. Computational Requirements: The system requires adequate memory and processing capacity to execute static analysis tools, perform CodeBERT inference, retrieve relevant security information, and operate the web application. Performance may be affected by large source files, lengthy functions, model configuration, and concurrent requests. A compatible GPU may improve inference performance, although CPU-based execution is also possible.

8. Remediation Requires Verification: The remediation suggestions generated by the system may be incomplete, unsuitable for the application's specific context, or introduce additional issues. Proposed changes must therefore be reviewed, tested, and re-scanned before being accepted. Automated verification, where implemented, can help identify remaining findings but does not guarantee that a correction is completely secure.

9. No Guarantee of Production Security: The proposed system is intended to assist developers with preliminary vulnerability detection and remediation. It is not a replacement for professional security audits, penetration testing, dynamic application security testing, manual code review, or established secure software development practices. Its findings should be considered alongside other security assessment methods before software is deployed in a production environment.

5.4 Applications

1. Secure Software Development: The system can be integrated into the software development workflow to analyze Python source code during development. It helps developers identify potentially insecure coding patterns early, allowing vulnerabilities to be addressed before the application is deployed.

2. Automated Code Security Review: The system can assist developers and security teams in conducting preliminary security reviews by combining static analysis tools such as Semgrep and Bandit with CodeBERT-based vulnerability classification. This reduces the amount of code that must be inspected manually during the initial review process.

3. Vulnerability Detection in Python Applications: The system can be used to analyze Python functions and identify potentially vulnerable code patterns. It is applicable to Python-based web applications, automation scripts, backend services, and other software components where identifying insecure code is important.

4. Developer Assistance and Secure Coding Education: The system can support students, programmers, and software development teams in understanding common security weaknesses. By providing vulnerability explanations and remediation suggestions, it helps users learn secure coding practices and recognize the potential consequences of insecure implementations.

5. Vulnerability Explanation and Classification: The Retrieval-Augmented Generation (RAG) component can retrieve relevant information from security knowledge resources, including the CWE Top 25 and OWASP Top 10. This information helps provide contextual explanations of detected issues, making the findings easier to understand and investigate.

6. Security Auditing and Code Assessment: Security analysts can use the system as an initial assessment tool when reviewing source code for potential vulnerabilities. The combined findings from static analysis and machine-learning classification can help prioritize code sections that require further manual investigation.

7. Remediation Planning and Code Improvement: The system can suggest possible corrections for identified security weaknesses. Developers can use these suggestions as a starting point for improving input validation, handling sensitive information, and addressing other insecure coding practices. Any proposed changes should be reviewed and tested before implementation.

8. Integration into Continuous Integration and Continuous Delivery (CI/CD) Workflows: The system's analysis components could be integrated into CI/CD pipelines to support automated security checks during code commits, pull requests, or build processes. Such integration would require appropriate pipeline configuration and should be considered an extension if it is not part of the current implementation.

9. Academic Research and Experimentation: The system can serve as a platform for studying the application of transformer-based models, static analysis, and retrieval-augmented generation in software security. It can also support experiments comparing detection approaches, evaluating model performance, and investigating the limitations of machine-learning-based vulnerability detection.

5.5 Chapter Summary

This chapter examined the advantages, limitations, and applications of the proposed vulnerability detection framework. The combination of static analysis, deep-learning-based classification, and security knowledge retrieval offers a comprehensive approach to identifying and understanding potential software vulnerabilities. The framework can assist developers in analysing security findings and obtaining relevant remediation guidance. However, its effectiveness depends on the quality and diversity of the dataset, the accuracy of model predictions, and the reliability of retrieved security information. Further development and evaluation can strengthen the integration of automated patch generation, patch verification, and the developer-facing interface. The framework has potential applications in secure code review, vulnerability assessment, and software development security workflows.

Chapter 6
Conclusion and Future Scope

6.1 Introduction

This chapter presents the overall conclusion of the project, *AI-Assisted Multi-Stage Framework for Source Code Vulnerability Detection*, and discusses the potential directions for future development. The project focuses on integrating traditional static application security testing, machine-learning-based vulnerability classification, Retrieval-Augmented Generation (RAG), and Large Language Models (LLMs) to support the identification, explanation, and remediation of potential software vulnerabilities.

The chapter summarizes the proposed framework, its major components, and the results obtained from the evaluation of the CodeBERT vulnerability classification model. It also discusses the significance of combining rule-based detection with contextual code analysis and AI-assisted security guidance. Furthermore, the chapter identifies opportunities to improve the framework through additional programming language support, enhanced dataset quality, improved model generalization, stronger patch verification, and integration with real-world software development workflows.

6.2 Conclusion

The project, *AI-Assisted Multi-Stage Framework for Source Code Vulnerability Detection*, aims to address the limitations of conventional vulnerability detection approaches by integrating multiple security analysis techniques into a unified workflow. Traditional Static Application Security Testing (SAST) tools can identify known insecure coding patterns but may produce false positives and provide limited remediation guidance. The proposed framework combines Semgrep and Bandit with a fine-tuned CodeBERT model to support the detection and classification of potentially vulnerable Python source code.

The framework incorporates the PyCode-Vul dataset for training and evaluating the vulnerability classification model. Dataset preprocessing is used to address missing and duplicate samples and reduce the risk of data leakage between dataset partitions. The classifier

is evaluated using accuracy, precision, recall, F1-score, and ROC-AUC to assess its performance under the selected evaluation conditions.

The reported evaluation results for the fine-tuned CodeBERT model include an accuracy of 96.88%, precision of 95.76%, recall of 97.38%, F1-score of 96.56%, and ROC-AUC of 99.33%. These results indicate promising classification performance on the evaluated test set. However, further testing on independent datasets and real-world code repositories is necessary to establish the model's generalization capability.

To provide contextual security guidance, the framework incorporates a Retrieval-Augmented Generation module using BGE-small-en-v1.5 embeddings and FAISS similarity search. The module retrieves relevant information from the MITRE Common Weakness Enumeration (CWE) knowledge base, which supports the generation of vulnerability explanations and potential remediation recommendations through an LLM. The framework also incorporates patch verification through rescanning modified code to assess whether previously identified security findings persist.

Overall, the project demonstrates the potential of combining static analysis, machine learning, and generative AI to assist developers in identifying and understanding software vulnerabilities. The integration of detection, explanation, remediation guidance, and verification provides a structured approach to vulnerability analysis. Nevertheless, the accuracy of the complete framework must be evaluated independently, and AI-generated recommendations and patches require appropriate human review and validation before implementation.

6.3 Future Scope

The proposed framework can be extended in several directions to improve its reliability, scalability, and practical applicability.

1. Support for Additional Programming Languages

The current vulnerability classification component primarily focuses on Python source code. Future development can extend the framework to languages such as Java, C, C++, and JavaScript by incorporating suitable datasets, language-specific security rules, and appropriate code models.

2. Improved Dataset Quality and Generalization

Future work can incorporate more diverse vulnerability datasets and strengthen preprocessing through exact and near-duplicate detection, leakage prevention, and project-level or chronological data splitting. Evaluation on independent datasets would help assess the model's ability to identify vulnerabilities in previously unseen code.

3. Advanced Vulnerability Detection

The detection pipeline can be enhanced by integrating additional static analysis tools, data-flow analysis, and interprocedural analysis. Experiments with alternative transformer models and ensemble techniques could also help determine whether detection performance can be improved.

4. Enhanced Retrieval-Augmented Generation

The cybersecurity knowledge base can be expanded to include additional CWE information, OWASP guidance, secure coding standards, and verified remediation examples. Improved retrieval ranking and document processing could provide more relevant information for generating accurate and context-aware explanations.

5. Stronger Patch Verification

The patch verification process can be extended beyond static rescanning by incorporating unit tests, integration tests, compilation checks where applicable, and security-specific test cases. These additional checks could help determine whether a proposed patch removes the vulnerability while preserving the intended functionality.

6. Integration with CI/CD Pipelines

The framework could be integrated into Continuous Integration and Continuous Deployment (CI/CD) pipelines to automatically analyze code during commits, pull requests, and software builds. This would support early vulnerability detection within existing software development workflows.

7. Performance and Scalability Improvements

Future versions could optimize model inference, introduce caching mechanisms, support batch processing, and use GPU acceleration where appropriate. These improvements could reduce processing time and support the analysis of larger source-code repositories and multiple concurrent users.

8. Explainability and Risk-Based Prioritization

The framework could provide more detailed explanations containing affected code locations, relevant CWE categories, supporting detection evidence, and remediation rationale. Vulnerabilities could also be prioritized using severity, confidence, and potential impact to help developers focus on the most significant risks.

9. Comprehensive Evaluation of the Integrated Framework

Further evaluation should examine the complete system using consistent datasets and protocols. In addition to classification metrics, assessment could include false-positive rates, false-negative rates, inference time, retrieval relevance, remediation correctness, and patch verification outcomes. This would provide a more complete understanding of the framework's practical effectiveness.

6.4 Chapter Summary

This chapter presented the conclusion and future scope of the AI-Assisted Multi-Stage Framework for Source Code Vulnerability Detection. The proposed framework integrates Semgrep and Bandit for static analysis, CodeBERT for vulnerability classification, RAG for retrieving relevant cybersecurity knowledge, and an LLM for vulnerability explanation and remediation guidance. It also incorporates patch verification through rescanning to assess whether previously detected security findings persist.

The reported CodeBERT evaluation achieved an accuracy of 96.88%, precision of 95.76%, recall of 97.38%, F1-score of 96.56%, and ROC-AUC of 99.33% on the evaluated test set. Although these results indicate promising classification performance, further independent evaluation is required to establish generalization and assess the effectiveness of the complete integrated framework.

The chapter also identified future improvements, including support for additional programming languages, stronger dataset validation, enhanced retrieval and remediation capabilities, more comprehensive patch verification, and integration with CI/CD pipelines. Overall, the project establishes a foundation for an AI-assisted software security tool designed to support developers throughout the vulnerability analysis process, while emphasizing the importance of human oversight and independent validation.

References

[1] A. Kharkar, R. Z. Moghaddam, M. Jin, X. Liu, X. Shi, C. Clement, and N. Sundaresan, "Learning to reduce false positives in analytic bug detectors," in *Proc. 44th Int. Conf. Software Engineering (ICSE)*, 2022.

[2] Y. Xiong and T. Zhang, "Sifting the noise: A comparative study of LLM agents in vulnerability false positive filtering," *arXiv preprint arXiv:2601.22952*, 2026.

[3] X. Du *et al.*, "Reducing false positives in static bug detection with LLMs: An empirical study in industry," in *Proc. ICSE 2026 (SEIP Track)*, arXiv:2601.18844, 2026.

[4] M. Iranmanesh, S. M. Sabet, S. Marefat, A. J. Ghasr, A. Wilson, I. Sharafaldin, and M. A. Tayebi, "ZeroFalse: Improving precision in static analysis with LLMs," *arXiv preprint arXiv:2510.02534*, 2025.

[5] Z. Jiang *et al.*, "DFEPT: Data flow embedding for enhancing pre-trained model based vulnerability detection," in *Proc. Internetware*, 2024, pp. 95–104.

[6] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, and M. Zhou, "GraphCodeBERT: Pre-training code representa-tions with data flow," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2021.

[7] Z. Xiong and W. Dong, "VulD-CodeBERT: CodeBERT-based vulnerability detection model for C/C++ code," 2024.

[8] Y. Jiang, Y. Zhang, X. Su, C. Treude, and T. Wang, "StagedVulBERT: Multigranular vulnerability detection with a novel pre-trained code model," *IEEE Trans. Software Engineering*, vol. 50, no. 12, 2024.

[9] R. Safdar *et al.*, "Data and context matter: Towards generalizing AI-based soft-ware vulnerability detection," *arXiv preprint arXiv:2508.16625*, 2025.

[10] Y. Chen *et al.*, "DiverseVul: A new vulnerable source code dataset for deep learn-ing based vulnerability detection," in *Proc. RAID*, 2023, pp. 654–668.

[11] Y. Ding *et al.*, "Vulnerability detection with code language models: How far are we?" In *Proc. ICSE*, 2025, pp. 469–481.

[12] L. Wartschinski *et al.*, "VUDENC: Vulnerability detection with deep learning on a

natural codebase for Python," *Inf. Softw. Technol.*, vol. 144, 106809, 2022.

[13] S-AiRLab, "PyCode-Vul: A benchmark dataset for Python," Kaggle dataset, 2026.

[14] X. Du *et al.*, "Vul-RAG: Enhancing LLM-based vulnerability detection via knowledge-level RAG," *arXiv preprint arXiv:2406.11147*, 2024.

[15] M. Fu *et al.*, "VulRepair: A T5-based automated software vulnerability repair," in *Proc. ESEC/FSE*, 2022. DOI: 10.1145/3540250.3549098.

[16] A. Al-Maamari, "Why LLMs fail: A failure analysis and partial success measurement for automated security patch generation," *arXiv preprint arXiv:2603.10072*, 2026.

[17] C. Cheng, "Detect-Repair-Verify for LLM-generated code: A multi-language, multi-granularity empirical study," *arXiv preprint arXiv:2603.23633*, 2026.

[18] Z. Feng *et al.*, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of EMNLP*, 2020.